

LYNX: Efficient Scheduling for Computation and Communication Overlapping in Large Model Training

Abstract

With the exponential growth of deep learning models and training datasets, distributed training has become increasingly indispensable. However, parallelism strategies necessary for training these large-scale models often suffer from suboptimal GPU utilization due to significant communication overheads in collective operations. Although computation-communication overlapping techniques aim to mitigate this, current methods rely on expert-crafted, model-specific computation-communication overlapping patterns, which are tightly coupled with specific distributed training systems, resulting in limitations in generality and performance.

We propose LYNX, a generic and efficient distributed training framework designed to optimize training performance by identifying optimal computation-communication overlapping patterns. LYNX formulates this optimization as a resource-constrained scheduling problem within a Directed Acyclic Graph (DAG), proposing a novel two-phase heuristic algorithm to rapidly and effectively identify optimal overlapping patterns. Additionally, LYNX employs subgraph rewriting techniques to efficiently manage frequent computation graph modifications and integrates a runtime feedback mechanism for accurate operator cost estimation. We implemented LYNX on top of XLA with comprehensive evaluations across multiple parallel strategies and model configurations. Our results demonstrate that LYNX achieves superior Model FLOPs Utilization (MFU) compared to the state-of-the-art frameworks such as Megatron-LM, significantly enhancing generality and performance in distributed large model training.

ACM Reference Format:

. 2025. LYNX: Efficient Scheduling for Computation and Communication Overlapping in Large Model Training. In . ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Recent advances in deep learning (DL) [17, 18, 23] have directly benefited from significant increases in both the size of the model parameters and the scale of the training data. For example, scaling transformer-based models, such as GPT-4 [5], to hundreds of billions of parameters and training them on much larger datasets has demonstrated remarkable language understanding and reasoning capabilities. Given that training these large-parameter models on massive datasets far exceeds the memory and compute limits of a single GPU, distributed model training has become indispensable.

However, current parallelism strategies for distributed training, such as tensor parallelism (TP) [43], sequence parallelism (SP) [30], fully sharded data parallelism (FSDP) [50],

and expert parallelism (EP) [25], face a critical challenge – *suboptimal GPU computational utilization*, which significantly delays the completion of end-to-end computations. For instance, Meta [14] reported that Model FLOPs Utilization (MFU) [12] fell below 43%. The primary factor contributing to suboptimal utilization is *communication*. During parallel training, collective operations [1], e.g., AllGather and ReduceScatter, are introduced in the critical computation path. These operations consume substantial runtime, during which most computational units remain idle. Therefore, efficiently utilizing idle computational units during communication is crucial for parallel training efficiency.

Computation-communication overlapping is widely employed to enhance GPU utilization and achieve efficient parallel training. Previous works have aimed to achieve efficient overlapping through three orthogonal and complementary approaches: (1) Finding the optimal overlapping pattern from existing ones, allowing as many communications as possible to execute concurrent computations that have no dependencies on them, thereby hiding more communication latency. (2) Instead of discovering existing optimal overlapping patterns that are difficult to identify, actively create opportunities that are easier to overlap. For example, Centauri [10] and other methods [20, 46, 47] decompose the original computation or communication operators into finer-grained units to create more opportunities. (3) Enhancing the parallel execution efficiency of computation and communication operators. Examples include kernel fusion techniques in CoCoNET [19], resource contention reduction approach in T3 [38], and communication offloading in ConCCL [6].

In this paper, we focus on finding the optimal overlapping patterns, based on our observation that this domain has not been fully explored. As summarized in Table 1, previous works have limitations in the following two aspects: (1) **Generality**: most works adopt algorithms specifically tailored for particular parallel strategies or models to discover overlapping patterns, or directly employ expert-crafted overlapping patterns, making their frameworks or algorithms lack generality. (2) **Performance**: most methods either find overlapping patterns from a fine-grained local view or focus solely on a coarse-grained global view, thereby overlooking numerous overlapping opportunities. As shown in Figure 1, the former explores intra-layer overlapping opportunities, neglecting inter-layer overlapping opportunities, while the latter treats each layer of DL models as the minimal units, overlooking the overlapping patterns within layers.

Recent advances in DL frameworks that incorporate compiler technology [4] (e.g., PyTorch [7] and XLA [2]) offer

WORK NAME	SUPPORTED PARALLELISMS	MODEL VARIETY	OPTIMIZATION VIEW	OPTIMIZATION ALGORITHM	FRAMEWORK GENERALITY
Centauri [10]	FSDP, DP, PP, TP	✗	MULTI-HIERARCHICAL	RULE-BASED ALGORITHM	↓
MegaScale [22]	DP, PP, TP	✗	FINE-GRAINED, LOCAL	EXPERT-CRAFTED	↓
Lancet [20]	EP	✗	FINE-GRAINED, LOCAL	RULE-BASED ALGORITHM	↓
Optimus [15]	DP, PP, TP	✓	MULTI-HIERARCHICAL	RULE-BASED ALGORITHM	↓
FOLD3D [27]	DP+PP+TP	✓	COARSE-GRAINED, GLOBAL	RULE-BASED ALGORITHM	↓
SimpleFSDP [48]	FSDP	✓	COARSE-GRAINED, GLOBAL	EXPERT-CRAFTED	—
XLA[2]	UNLIMITED	✓	FINE-GRAINED, GLOBAL	RULE-BASED ALGORITHM	↑
LYNX (our solution)	UNLIMITED	✓	FINE-GRAINED, GLOBAL	HEURISTIC ALGORITHM	↑

Table 1. Comparison of Different Works. SUPPORTED PARALLELISM indicates the parallel strategies supported by the framework, where DP+PP+TP means only this hybrid parallel strategy is supported, while DP,PP,TP indicates that the framework supports both individual parallel strategies and their hybrid parallel. MODEL VARIETY refers to whether the framework supports various model structures. OPTIMIZATION VIEW (§2.2) refers to the optimization view of the framework. OVERLAPPING ALGORITHM (§2.2) refers to the algorithm for finding the optimal overlapping patterns.

the possibility of discovering more overlapping opportunities from a finer-grained global view, while maintaining the inherent generality of these frameworks. This becomes possible because these frameworks organize the execution steps of DL models as a computation graph, specifically a Directed Acyclic Graph (DAG), where nodes represent computation or communication operators, and edges represent the data dependencies between them. Consequently, we can identify computations and communications without dependencies in the DAG, allowing for their overlapped execution.

However, it remains *non-trivial* to find the optimal overlapping pattern from the DAG, mainly due to the following factors. *First*, the DAG contains a large number of nodes and edges with complex dependency relationships, constituting an extensive search space (e.g., training a 7B model involves 10,000 nodes), thus requiring an efficient algorithm capable of quickly finding the optimal overlapping pattern. *Second*, the DAG frequently changes (e.g., stochastic sampling alters the computation workflow), necessitating frequent recomputation of overlapping patterns. *Finally*, the cost estimation of each operator in the DAG affects the results of finding the optimal overlapping pattern. Inaccurate cost estimation may lead to suboptimal or even erroneous overlapping patterns.

To address these challenges, we introduce LYNX, a **generic** and **efficient** distributed training system designed to optimize the training efficiency of DL models by hiding communication latency by finding the optimal overlapping patterns. LYNX first models the problem of finding optimal overlapping patterns in a DAG as a *resource-constrained scheduling* problem, which is NP-hard [41]. Then, based on the analysis of the distribution of communication and computation operators in the DAG under various parallel strategies, we propose a *two-phase heuristic algorithm* that can find a more

optimal overlapping pattern in less time. Furthermore, LYNX introduces *subgraph rewrite* techniques to mitigate overhead caused by frequent computation graph modifications and integrates a *runtime feedback* mechanism that can more accurately estimate operator costs in various scenarios.

We implemented LYNX in C/C++, Python, and Java across 10k lines of code based on XLA [2]. The implementation includes converting code into the resource-constrained scheduling problem, heuristic algorithms for solving the problem, and functionalities of subgraph annotation and subgraph rewrite. We also leveraged SQLite to store and estimate operator costs, enabling runtime feedback mechanisms. Experiments demonstrate that LYNX achieves superior MFU compared to Megatron-LM [43], a mainstream training system that employs expert-crafted overlapping patterns, and XLA [2], the state-of-the-art deep learning compilation framework, across various scenarios. Furthermore, experiments show that LYNX achieves performance improvements across various parallel strategies (including single parallel strategies and hybrid parallel strategies) as well as various models, showcasing its generality. In summary, we make the following contributions:

- We frame the problem of finding the optimal overlapping patterns in DAG as a resource-constrained scheduling problem and propose a two-phase heuristic algorithm to obtain better solutions with less time.
- We present a subgraph rewrite method to reduce the computational cost of resolving the scheduling problem caused by frequent DAG changes.
- We introduce a runtime feedback mechanism to more accurately estimate operator costs.
- We evaluate LYNX thoroughly to show its generality and performance compared to the state-of-the-art.

2 Background and Motivation

2.1 Distributed Training

Training large-parameter models on massive datasets requires memory and computing that far exceeds the capacity of a single GPU. Therefore, distributed parallel training becomes indispensable with a variety of parallel strategies each exhibiting unique communication patterns.

Data Parallelism (DP) [31] distributes training samples across multiple devices and accumulates gradients from different devices through AllReduce. Fully Sharded Data Parallelism (FSDP) [50] combines DP and ZeRO-3 [19] techniques. ZeRO-3 maintains shards of model parameters, gradients, and optimizer states on each device. During computation, these shards are dynamically collected from other devices via AllGather and released after use. During backpropagation, ReduceScatter accumulates and distributes gradients shards. Tensor Parallelism (TP) [43] partitions model weights across multiple devices, similar to partitioned matrix computation, where each device is only responsible for a portion of the computation. These partial results are then collected and aggregated through AllReduce or AllGather. Expert Parallelism (EP) [25] dynamically dispatches computational tasks to specialized experts across different devices via AllToAll.

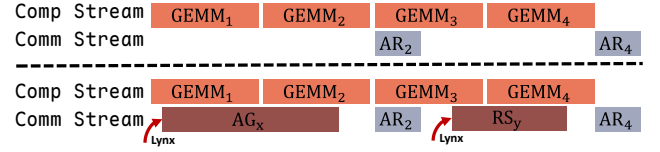
In general, these parallel strategies introduce communication (via collective operations) along critical computation paths. These communications incur significant runtime overhead (e.g., as shown in Figure 7(a), typically accounting for 50% of the total latency) and leave computational units idle during communication, resulting in low utilization of computational resources and delaying overall completion.

2.2 Overlapping Strategies and Their Limitations

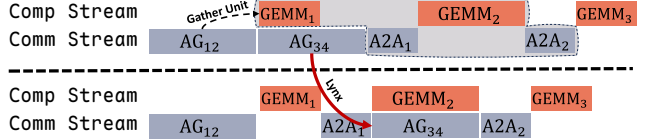
Previous research aimed to find overlapping patterns to simultaneously execute as many independent computation and communication operators as possible, thus hiding the latency induced by communication. However, as illustrated in Table 1, these studies demonstrate two key limitations.

Generality: Many frameworks or proposed algorithms only explore overlapping patterns within specific model architectures or parallel strategies, limiting their generality. For example, Centauri [10] and MegaScale [22] exclusively support Transformer-based models; Lancet [20] focuses solely on Mixture-of-Experts [42] models with specialized structures; and SimpleFSDP [48] restricts itself to FSDP. Moreover, certain frameworks embed expert-crafted overlapping patterns into their systems, such as MegaScale [22], which *hardcodes* specific overlapping patterns into the frameworks.

Performance: Current approaches to identifying overlapping patterns have notable limitations in their optimization views. Methods such as MegaScale [22] and Lancet [20] adopt a fine-grained *local* view, neglecting the global view. As illustrated in Figure 1(a), in the TP+ZeRO hybrid parallel scenario, these methods focus on overlapping in the TP-related local context



(a) The fine-grained local optimization view (TP + ZeRO). GEMM represents General Matrix Multiplication, AR_n denotes performing AllReduce on the results of $GEMM_n$, AG_x represents using AllGather to collect parameters required for the x -th operator, and RS_y represents performing ReduceScatter on the results of the y -th operator.



(b) The coarse-grained global optimization view (EP + FSDP). $A2A_n$ represents using AllToAll to the dispatch or collect of $GEMM_n$ results, while AG_{ij} represents using AllGather to collect parameters required by the ij -th computational units.

Figure 1. Computation-communication overlapping under local (a) and global (b) optimization view.

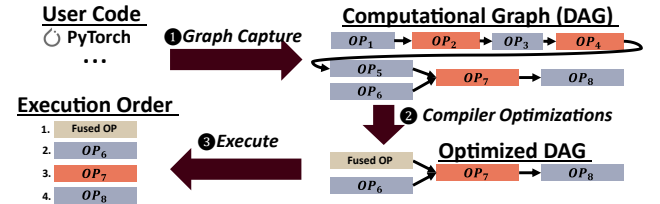


Figure 2. Workflow of machine learning compilation.

while ignoring communication in ZeRO that have no dependencies with TP, such as AllGather and ReduceScatter used for gathering and updating parameters or gradients, whose computation will occur later.

Conversely, methods like FOLD3D[27] and SimpleFSDP[48] adopt a coarse-grained *global* view, neglecting local details. As shown in Figure 1(b), under a global view, these methods overlap the AllGather for gathering parameters for the next computation unit with the current computation, overlooking the communication pattern within the computation unit. This results in suboptimal overlapping patterns, whereas delaying the execution of the AllGather could yield more efficient overlapping patterns.

Furthermore, Centauri [10] and Optimus [15] employ a multi-hierarchical optimization approach, finding overlapping patterns from a coarse-grained global view first and then fine-grained local view. Although this greedy rule-based algorithm achieves certain performance improvements, in hybrid parallel scenarios with complex dependencies, the near-optimal overlapping patterns at each hierarchy do not necessarily constitute a globally optimal overlapping pattern.

2.3 Machine Learning Compiler

Under the two optimization views, modeling an efficient algorithm to find the optimal overlapping patterns is extremely challenging. Modern DL frameworks, when augmented with machine learning (ML) compiler techniques [11, 28], construct a Direct Acyclic Graph (DAG) that offers a global, fine-grained view. As shown in Figure 2, the ML compiler analyzes the dependencies among all execution steps of the DL model (*global view*), transforms them into a DAG, where nodes represent fine-grained execution units, i.e., operators, and edges represent the data dependencies between them (*fine-grained view*). This explicit graph representation enables advanced cross-operator optimizations such as operator fusion across the model. After these optimizations, the compiler typically generates an execution schedule by performing a topological traversal of the DAG to enforce dependency ordering. Finally, these operators are executed *one by one* according to the schedule, i.e., topological sorting of the DAG.

However, executing the DAG in strict topological order imposes a largely sequential schedule that fails to exploit much of the available parallelism. For example, XLA [2] attempts a limited form of overlap by greedily scheduling a communication operation in parallel with a subsequent compute operation if no dependencies exist. This local, greedy strategy – confined to a sequential schedule – often yields suboptimal global results.

2.4 Motivation and Challenges

To overcome such limitations, we propose framing the search for an optimal computation-communication overlapping pattern as a *resource-constrained scheduling problem*, rather than relying on a single linear ordering.

We view the GPU as a key-constrained resource that can only process two different types of operators, i.e., computation and communication, simultaneously without interruption at any time. When the GPU becomes idle, operators that satisfy dependencies (i.e., all their predecessor operators in the DAG have been executed) compete for this limited resource. This necessitates a scheduling strategy to determine which operator should be executed on the GPU resource. Our objective is to find the optimal scheduling strategy to determine the scheduling order of operators, minimizing the overall scheduling latency. However, determining the scheduling order of operators for DL execution represented as a DAG poses two main **challenges**.

Scheduling Complexity: The general scheduling problem – allocating n operators with precedence constraints, where each operator n_i has an execution time $W(n_i)$, to k processors for uninterrupted execution, with the objective of finding the execution order that minimizes the total completion time – was rigorously proven to be NP-complete by Ullman in 1975 [45]. Our resource-constrained scheduling problem involves even more complex constraints – treating the GPU as

two *processors*¹, one exclusively for computation-type operators and one exclusively for communication-type operators. Therefore, according to computational complexity theory, our problem is at least NP-hard. Moreover, our problem involves complex dependencies, for example, in the DAG for training a 7B model, there exist more than 10,000 nodes and edges. Consequently, obtaining an optimal solution to this NP-hard problem within practical time constraints is hard.

Modeling Problem Complexity: Although current mainstream DL frameworks support representing DL execution steps as a DAG in the form of intermediate representations (IRs), it remains difficult to obtain the execution time (or cost) $W(n_i)$ for each operator n_i , which is crucial for finding optimal solutions to the scheduling problem. An inaccurate $W(n_i)$ value may lead to suboptimal or even incorrect scheduling orders. Therefore, it is necessary to accurately estimate the cost of each operator in the DAG, especially in scenarios where identical operators may have different costs across different cluster environments. Moreover, the DAG will change during the execution of DL models. For example, the Mixture-of-Experts (MoE) model execution involves random sampling operators, which lead to the execution of different control flow branches. Consequently, after changes in the DAG, it becomes necessary to resolve new overlapping patterns, which further extend the end-to-end completion latency.

3 Design of LYNX

In this section, we first present an overview of LYNX, covering its key design and workflow, followed by a detailed explanation of the core techniques that underpin the system.

3.1 Overview

To address those challenges (§2.4), we designed and built a generic and efficient distributed training system, LYNX. As shown in Figure 3, LYNX consists of three main components: ❶ SubGraph Rewrite (§3.4), which employs a special *custom-call* operator to represent potentially frequently varying subgraph portions annotated by domain experts during DL model execution, thereby reducing the overhead of resolving optimal overlapping patterns across the entire graph. ❷ Efficient heuristic scheduling algorithm (§3.2), which is a two-phase heuristic scheduling algorithm based on our observation and analysis of the distribution of computation and communication operators in the computational graphs (DAG) under various parallel strategies. ❸ Runtime Feedback

¹To maintain consistency with general scheduling problem descriptions, we abstract the GPU as two *processors*, one representing the stream for communication, and the other representing the stream for computation, which can execute operators simultaneously. As discussed in §5, although a GPU supports multiple streams executing multiple different types of operators concurrently, our work primarily focuses on scenarios where a single communication operator overlaps with a single computation operator.

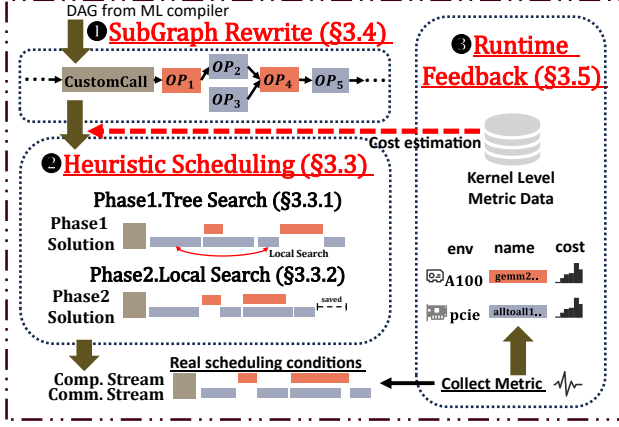


Figure 3. Overview of LYNX.

(§3.5), which collects kernel-level operator performance metrics from different cluster environments to provide accurate operator cost estimates for the scheduling algorithm.

Specifically, LYNX obtains the DAG representing model execution steps from DL frameworks with integrated compiler technology and identifies subgraph regions annotated by experts, replacing them with a special *custom-call* operator. Subsequently, a two-phase heuristic scheduling algorithm is employed to determine the scheduling order of these operators, whose dependencies are represented by the DAG. Following the scheduling order, we dispatch computation and communication operators to the corresponding execution streams for execution. During execution, we continuously collect performance metrics of these kernel-level operators through profiling tools and use these metrics to guide subsequent optimization processes.

3.2 Resource-Constrained Scheduling Model

To find the optimal overlapping pattern, LYNX models this problem as a resource-constrained scheduling problem.

First, LYNX converts the DL model’s execution steps into a DAG data structure to more flexibly describe dependency constraints between operators. Specifically, DL frameworks typically use an ordered instruction set $\mathcal{I}$ to describe model execution steps, where each instruction $i \in \mathcal{I}$ takes the form: $r = op(o_1, \dots, o_n)$, with r being the result obtained by performing operator op using operands $o_1, \dots, o_n$. While this linear form is suitable for execution, it hinders efficient dependency analysis. LYNX addresses this by transforming the ordered instruction set $\mathcal{I}$ into a DAG, through Algorithm 1, represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of all nodes (i.e., operator op) in the graph. Each node has an execution cost, $W(v), v \in \mathcal{V}$, estimated by the runtime feedback component (described in §3.5). $\mathcal{E}$ is the set of all directed edges in the graph, where a directed edge $e = \langle u, v \rangle, e \in \mathcal{E}$, describes the dependency relationship between operators, i.e., v must execute after u , where $u, v \in \mathcal{V}$.

Therefore, finding the optimal overlapping pattern – i.e., enabling as many communications as possible to execute simultaneously with computations that have no dependencies on them – can be transformed into the following resource-constrained scheduling problem: Given a set $\mathcal{V}$ of operators (or nodes) ready to be executed, which are non-preemptible during execution, minimize the end-to-end scheduling latency under two key critical constrained resources (namely, one *processor* for executing computation and one *processor* for executing communication), while satisfying a series of constraints $\mathcal{C}$. The constraints $\mathcal{C}$ include two conditions, as shown in Equation 1: C_1 requires that the start time $s(v)$ of any node $v \in \mathcal{V}$ must occur after all its predecessor nodes have been executed. C_2 requires that for any two nodes v_i and v_j , if they require the same type of *processor* $T(v_i)$ and $T(v_j)$, they cannot be executed simultaneously (i.e., only a computation and a communication operator can overlap).

$$C_1 : \forall e \in \mathcal{E}, s(e.v) \geq s(e.u) + W(u) \quad (1)$$

$$C_2 : \forall v_i, v_j \in \mathcal{V}, v_i \neq v_j, T(v_i) = T(v_j) \Rightarrow [s(v_i), s(v_i) + W(v_i)) \cap [s(v_j), s(v_j) + W(v_j)) = \emptyset$$

As a result, finding the optimal overlapping patterns means determining the start time for all nodes under constraint $\mathcal{C}$, such that the value of the objective Equation 2 is minimized. After determining the start time for each node, we sort these nodes by start time. Then, based on their operator types, launch them one by one in the sorted order to execute on the corresponding *processor*.

$$O = \max\{s(v) + W(v) | v \in \mathcal{V}\} \quad (2)$$

Existing Solution: Integer Linear Programming. We can formulate the aforementioned resource-constrained scheduling problem as an Integer Linear Programming (ILP) [13] $\mathcal{F}$, as shown in Equation 3. Here, $\mathcal{F}_1$ is identical to C_1 , and $\mathcal{F}_2$ is the linear inequality representation of C_2 . Our objective is to find non-negative integer solutions for n variables $s(v_1), \dots, s(v_n)$, where $v_1, \dots, v_n \in \mathcal{V}$ and $(n-1)!$ binary solutions $b_1^2, \dots, b_n^{n-1}$, such that the value of O is minimized. Algorithm 1 describes how to formulate such a scheduling problem into an ILP. This ILP problem can then be solved using off-the-shelf mathematical solvers, such as Google’s OR-Tools [39] solver.

$$\mathcal{F} = \begin{cases} \mathcal{F}_1 : \forall e \in \mathcal{E}, s(e.v) - s(e.u) \geq W(u) \\ \mathcal{F}_2 : \forall v_i, v_j \in \mathcal{V}, i > j, T(v_i) \neq T(v_j), \\ \quad s(v_i) - s(v_j) > -\infty b_i^j \\ \quad s(v_j) - s(v_i) > -\infty(1 - b_i^j) \end{cases} \quad (3)$$

Observation and Analysis. Employing off-the-shelf mathematical solvers to find optimal overlapping patterns is feasible; however, when applying these solvers to large-scale models comprising tens of thousands of nodes and complex constraints, as illustrated in Figure 10 of our experiment,

Algorithm 1: Formulate the IR as an Integer Linear Programming (ILP).

Input: IR $\mathcal{I}$
Output: ILP $\mathcal{F}$

```

1  $\mathcal{V} \leftarrow \emptyset, \mathcal{E} \leftarrow \emptyset, \mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$ 
2 /* convert the IR into DAG */
3 for  $i \in \mathcal{I}$  do
4    $v \leftarrow i.op, \mathcal{V} \leftarrow \mathcal{V} \cup \{v\}$ 
5   for  $operand\ o \in i.operands$  do
6      $u \leftarrow o.op, \mathcal{V} \leftarrow \mathcal{V} \cup \{u\}, \mathcal{E} \leftarrow \mathcal{E} \cup \{<u, v>\}$ 
7   end
8 end
9 /* ILP Formulation */
10  $\mathcal{F} \leftarrow InitILP()$ 
11 for  $e \in \mathcal{E}$  do
12    $\mathcal{F}.Var(s(e.u) \geq 0), \mathcal{F}.Var(s(e.v) \geq 0)$ 
13    $\mathcal{F}.Constraint(s(e.v) - s(e.u) \geq W(u))$ 
14 end
15 for  $v_i, v_j \in \mathcal{V}, i > j, T(v_i) \neq T(v_j)$  do
16    $\mathcal{F}.Var(b_i^j \in \{0, 1\})$ 
17    $\mathcal{F}.Constraint(s(v_i) - s(v_j) \geq -\infty b_i^j)$ 
18    $\mathcal{F}.Constraint(s(v_j) - s(v_i) \geq -\infty(1 - b_i^j))$ 
19 end
20 return  $\mathcal{F}$ ;

```

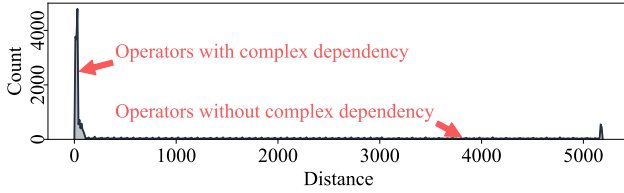


Figure 4. Distribution characteristics of operators in computational graphs. Distance refers to the number of nodes traversed from an operator node to the *exit* node in a DAG, where the *exit* node is special to which all nodes without successors point.

obtaining satisfactory solutions within a limited time using ILP-based approaches often proves challenging.

To address this, we adopt *search-based* scheduling algorithms, which can leverage problem-specific knowledge and heuristic methods to guide the scheduling process (i.e., thereby reducing the search space). Similar to traditional scheduling algorithms, we schedule operators according to their dependencies, represented by a DAG. However, at each step, multiple operators may simultaneously satisfy the scheduling constraints, while we have only one processor for computation and one for communication. As only one computation or communication operator can execute at a time, determining the execution order among these ready operators is critical.

To solve this, we employ a priority-based approach to select the most important operator from the available candidates. To define this priority, we analyzed the distribution of computation and communication operators in the computational graph, as shown in Figure 4. We observed that

the distance between operator nodes and the exit node follows a long-tail distribution, and operators in the head and tail regions exhibit different dependency characteristics. Accordingly, our proposed priority formulation consists of two components, each designed to capture features from these distinct regions, as shown in Equation 4.

$$Prio(v) = rank_u(v) - eft(v) \quad (4)$$

Part 1. Operators with no predecessors or few dependencies can be executed as early as possible to pre-fetch or compute data needed by subsequent operators, such as AllGather in ZeRO or FSDP strategies. To determine the priority of such operators, we use the *upward rank* advocated in HEFT [44], as calculated in Equation 5, where $succ(v)$ represents the successor nodes of node v , and $W(v)$ represents the execution cost of node v . The $rank_u$ value reflects the impact of an operator on the *global* execution time; the larger its value, the greater its influence on subsequent operators, making it more likely to be considered a critical operator (i.e., to be scheduled earlier).

$$rank_u(v) = W(v) + \max_{u \in succ(v)} (rank_u(u)) \quad (5)$$

Part 2. Operators with complex predecessor dependencies often lie on long execution paths, such as gradient aggregation operators like AllReduce. These operators typically have similar $rank_u$, indicating that they are close to the *exit* node. We adopt the dynamic Earliest Finish Time (EFT) to consider the priority of these operators. Its calculation is shown in Equation 6, where $pred(v)$ represents the predecessor nodes of node v . When calculating eft , since computation and communication operators can only be executed on their corresponding *processor*, it is also necessary to consider the earliest available time of the *processor* required by node v , denoted as $AT(v)$; unlike $rank_u$, which can be calculated offline, $AT(v)$ needs to be updated during the search process. The eft reflects the earliest possible finish time of a node – the smaller its value, the more likely it is to greedily provide sufficient time for subsequent operators.

$$\begin{aligned}
 eft(v) &= est(v) + W(v) \\
 est(v) &= \max \left(\max_{u \in pred(v)} (est(u) + W(u)), AT(v) \right) \quad (6)
 \end{aligned}$$

In summary, the priority, as defined in Equation 4, simultaneously considers the globally-focused $rank_u$ and the locally-focused eft based on a greedy approach. The higher this value, the earlier it should be scheduled.

Our Solution: Two-phase Heuristics Algorithm. While the priority defined in Equation 4 reduces the search space, relying on it alone may overlook complex dependencies and result in scheduling decisions that are locally optimal but globally suboptimal. To overcome this, we propose a *two-phase heuristic scheduling algorithm* (§3.3). In the first phase,

Algorithm 2: Tree Search Algorithm

Input: DAG $\mathcal{G}(\mathcal{V}, \mathcal{E})$
Output: Schedule $\mathcal{S}$

```
1  $R \leftarrow \text{ComputeRank}_u(\mathcal{G})$  // Equation 5
2  $U \leftarrow \mathcal{G}.\mathcal{V}$  // Unscheduled nodes
3 // Computation and Communication available time
4  $A \leftarrow \{0, 0\}$ 
5 while  $U \neq \emptyset$  do
6    $R \leftarrow \{v | v \in U, \text{pred}(v) \subseteq (\mathcal{V} - U)\}$ 
7   for  $t \in \{\text{comp}, \text{comm}\}$  in parallel do
8     if  $\text{NotAvailable}(A(t))$  then
9       continue
10    end
11     $R_t \leftarrow \{v | v \in R, T(v) = t\}$ 
12    // The TopK from  $R_t$  using Equation 4
13     $P \leftarrow \text{TopK}(R_t, \text{Prio})$ 
14     $C \leftarrow \emptyset$  // Simulation scheduling results
15    for  $p \in P$  in parallel do
16       $C \leftarrow C \cup \{\text{SimSchedule}(c)\}$ 
17    end
18    // The min from  $C$  using Equation 7
19     $t \leftarrow \text{Min}(C, \text{ratio})$ 
20    // Schedule and update available time
21     $\mathcal{S} \leftarrow \mathcal{S} \cup \text{Schedule}(t, A)$ 
22     $U \leftarrow U - \{t\}$ 
23  end
24 end
25 return  $\mathcal{S}$ 
```

we employ the *Tree Search* algorithm that evaluates the impact of current scheduling decisions on global scheduling results through a multi-step look-ahead method, which constructs an initial scheduling plan from a global view. In the second phase, we use the *Local Search* algorithm to adjust local regions within the first phase solution to explore better overlapping patterns in localized areas.

3.3 Heuristic Scheduling Algorithms

Tree Search Algorithm. To avoid suboptimal scheduling decisions caused by relying solely on the greedy priority from Equation 4, we adopt a multi-step look-ahead *Tree Search* algorithm, as shown in Algorithm 2.

This algorithm simulates scheduling for each operator that is ready for execution, evaluating how current choices affect global performance across multiple future steps. It then selects the operator whose simulation yields the best outcome as the next node to schedule. However, simulating every ready operator at each iteration is computationally infeasible due to the large number of nodes and dependencies. To address this, we limit the search to the top- K ready operators ranked by priority (*Prio*), and simulate scheduling paths of depth d , corresponding to d iterations of future scheduling decisions. These simulations use only the priority-based strategy from Equation 4 to maintain efficiency.

Algorithm 3: Local Search Algorithm

Input: First Phase Schedule $\mathcal{S}$
Output: Second Phase Schedule $\mathcal{S}'$

```
1  $\mathcal{S}' \leftarrow \mathcal{S}$ 
2 for  $i$  to  $I$  do
3    $p \leftarrow \text{False}, C \leftarrow \text{NonOverlappingComm}(\mathcal{S})$ 
4   for  $u \in C$  do
5     for  $v \in \text{NeighborComm}(u)$  do
6       // adjust the order of operators
7        $\mathcal{S}'' \leftarrow \text{Adjust}(\mathcal{S}, u, v)$ 
8       // compare the latency
9       if  $\mathcal{S}'' < \mathcal{S}'$  then
10         $\mathcal{S}' \leftarrow \mathcal{S}'', p \leftarrow \text{True}$ 
11      end
12    end
13    // adjust the local can achieve improvement
14    if  $p = \text{True}$  then
15      break
16    end
17  end
18  // adjustment can not achieve improvement
19  if  $p = \text{False}$  then
20    break
21  end
22 end
23 return  $\mathcal{S}'$ 
```

To evaluate each candidate operator v , we use Equation 7, where $T_{\text{sim}}(v)$ is the simulated end-to-end latency after overlapping communication and computation, and $C_{\text{sim}}(v)$ is the cumulative cost assuming no overlap (i.e., pure sequential execution). A smaller $\text{ratio}(v)$ indicates a greater degree of overlapping and, therefore, better scheduling efficiency. The operator with the minimum ratio is selected as the next scheduling decision.

$$\text{ratio}(v) = \frac{T_{\text{sim}}(v)}{C_{\text{sim}}(v)} \quad (7)$$

Time Complexity of Tree Search. Assuming the problem contains n nodes, we need to traverse and schedule n nodes. During the scheduling process, the time complexity of priority calculation is $O(n)$, the time complexity of the top- K algorithm is $O(K \log k)$, and we perform k simulation schedules, each with a time complexity of $O(n)$. Therefore, the time complexity is: $O(n) \cdot [O(n) + O(K \log k) + O(kn)] = O(n^2)$

Local Search Algorithm. The first-phase *Tree Search* algorithm efficiently explores the global search space to identify a high-quality initial schedule. However, it may overlook fine-grained overlapping opportunities at the local level due to its global focus. To address this, we introduce a second-phase *Local Search* algorithm, as shown in Algorithm 3, which targets local regions of the schedule and iteratively refines them to improve communication-computation overlap.

The key steps of the *Local Search* algorithm are as follows: ① Identify communication operators that do not currently overlap with computation, and for each, locate nearby dependency-free communication operators (i.e., those scheduled immediately before or after it in the first-phase schedule). ② Swap the execution order of the two communication operators and adjust the order of other affected operators to preserve the original dependency constraints defined by the DAG. ③ Evaluate the updated schedule to determine whether the adjustment improves performance, specifically, whether it increases overlap and reduces end-to-end latency.

Time Complexity of Local Search. We need to iterate I times. In each iteration, finding non-overlapping communication operators has a time complexity of $O(n)$. When traversing each non-overlapping communication operator, searching its neighborhood requires $O(w)$ time complexity, where w is a constant parameter. Adjusting the scheduling order has a time complexity of $O(n)$. Therefore, the final time complexity is: $O(I) \cdot [O(n) + O(n) \cdot O(w) \cdot O(n)] = O(In^2w) = O(n^2)$. The time complexity of both phase is the same, therefore the overall time complexity is also $O(n^2)$.

Summary. To facilitate both global planning and local refinement, the two-phase heuristic algorithm in the first phase constructs an initial schedule by simulating future decisions guided by priority and overlapping efficiency, while in the second phase refines this schedule by locally adjusting operator orderings to enhance overlap. Together, these phases enable efficient, scalable, and high-quality scheduling with an overall time complexity of $O(n^2)$.

3.4 SubGraph Rewrite

To address the issue of frequent DAG changes due to execution across different control flow branches as discussed in §2.4, we propose a *subgraph rewrite* technique. It minimizes the overhead associated with repeatedly optimizing similar computational subgraphs encountered during dynamic execution paths. The works as follows (in Figure 5). ① Domain experts annotate subgraphs that require rewrite in the entire computational graph. ② Verify if the subgraphs have undergone overlapping optimization. ③ When a subgraph has not undergone overlapping optimization, the heuristic search algorithm is applied to optimize it. Additionally, we utilize off-the-shelf compilation techniques (e.g., operator fusion) for further optimization. The optimized subgraph is then cached. ④ The subgraph is replaced with a special operator (i.e. *custom-call*) that represents the subgraph.

SubGraph Annotate and Rewrite. LYNX employs expert annotation rather than automatic identification methods to partition subgraphs. This choice is motivated by the ease with which domain experts, during model development, can precisely pinpoint minimal subgraph units whose operators or computational workflows may vary at runtime. Experts are also better positioned to accurately label these subgraphs

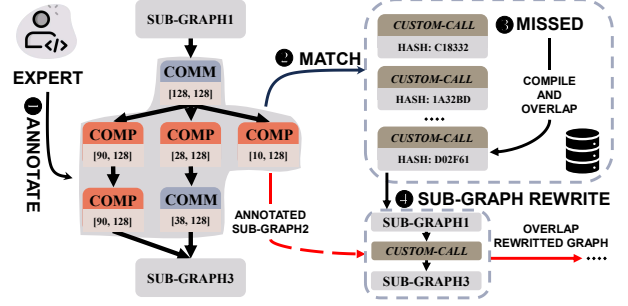


Figure 5. Overview of Graph Rewrite.

regarding whether they involve computation, communication, or both. After annotation, LYNX converts these expert-labeled dynamic subgraphs into special invariant *custom-call* operators. Subsequently, our proposed heuristic algorithm is applied to optimize the entire computational graph with these substitutions in place. Since runtime changes within these subgraphs do not affect the overall computational graph structure, it is unnecessary to recompute global overlapping patterns. Instead, only the overlapping patterns within the modified subgraphs require recalculation.

SubGraph Match. LYNX employs a caching mechanism for subgraphs after overlapping optimization to further reduce optimization overhead. It leverages hash functions to generate unique identifiers based on the preorder traversal sequences of subgraphs. This method detects changes in annotated subgraph regions and quickly selects the appropriate cached subgraphs for execution when the *custom-call* operator is invoked. Additionally, during the execution of DL models on GPU, LYNX proactively enumerates potential subgraph variants in the background when the CPU resources required for finding the optimal overlapping pattern are idle, performing proactive optimization and caching, which further hides finding overlapping pattern overhead and enhances runtime performance.

3.5 Runtime Feedback

When using the proposed heuristic scheduling algorithm to identify optimal overlapping patterns, the value of $W(u)$ in Equation 4, which estimates the execution time of node u , substantially affects the final solution – inaccurate estimations can lead to suboptimal or even incorrect solutions. This accuracy is particularly important in distributed environments, where identical computation and communication operators may incur varying execution costs depending on the specific hardware and device conditions.

To this end, during DL model execution, LYNX leverages performance profiling tools, such as NVIDIA’s CUDA Profiling Tools Interface (CUPTI) [3], to monitor kernel-level operator execution costs across various device environments. These performance metrics are recorded into a database, categorized by device configuration and operator characteristics,

enabling accurate cost estimations during optimization. By default, after excluding outliers from collected data, we utilize the mean execution cost as our estimation value.

LYNX also tracks the estimated cost of each operator. If it detects significant discrepancies with the estimates used in previous solutions, it will solve for new overlapping patterns using the heuristic algorithm. Notably, this solving process does not interrupt the execution of the DL model; the replacement only occurs in the next iteration when the computational graph is executed again after a new overlapping pattern has been solved in the background. This approach further hides the time required to solve optimal overlapping patterns by overlapping the DL model execution with the pattern-solving process.

4 Evaluation

LYNX has been successfully deployed in our production environment to optimize the deep learning model training code submitted by our data scientists. LYNX’s generality makes the optimization process completely transparent to the data scientists. Further, LYNX effectively reduces communication overhead through computation and communication overlapping, significantly improving GPU utilization, reducing training time for our in-house Transformer-based LLMs by 18% and multi-modal large models by 25%.

In this section, we demonstrate the effectiveness of LYNX by first evaluating its end-to-end performance and scalability, and then examining the benefits of the heuristic algorithm, subgraph rewrite, and runtime feedback individually.

4.1 Experimental Setup

Platforms. Our experimental platforms consist of both single-machine and multi-machine setups. In the single-machine setup, we use $8 \times$ NVIDIA H800 GPUs (each with 80GB memory), interconnected via 400GB/s NVLink. In the multi-machine setup, each machine contains $8 \times$ NVIDIA H800 GPUs (each with 80GB memory), with intra-machine GPUs connected via 400GB/s NVLink, while inter-machine communication occurs through the 400Gb/s network adapter. Each GPU connects to the network adapter via a 63GB/s PCIe 5.0 x16 interface, consequently limiting inter-machine communication to PCIe speeds. Except for scalability experiments, all other experiments use a maximum of 2 machines. For scalability experiments, we employ eight machines (i.e., 8-node), each equipped with $8 \times$ NVIDIA H200 GPUs (each with 141GB memory). Within these nodes, GPUs are interconnected via 478GB/s NVLink, while inter-node bandwidth is identical to that in the multi-machine setup.

Models and Workloads. For our evaluation, we select four representative language models with different parameter scales and architectures to demonstrate LYNX’s performance. ModelA and ModelB are Transformer architecture models with different parameter scales, while ModelC and ModelD

are models based on Mixture-of-Experts (MoE) architecture, whose detailed configuration is shown in Table 2. In all the experiments, we set the sequence length to 4096 and adjusted the batch size to the maximum feasible size to test the performance of LYNX.

Table 2. Models For Evaluation

Name	Parameter	Hidden Size	Layers
ModelA	7B	4096	32
ModelB	13B	5120	40
ModelC	25B	4096	32
ModelD	49B	5120	40

Baselines. We compare LYNX with two state-of-the-art baseline methods: (1) *Megatron-LM*[43] is a PyTorch-based distributed training framework that integrates tensor parallelism (TP), data parallelism (DP), and expert parallelism (EP). It implements efficient computation-communication overlapping patterns for some fixed hybrid parallelism strategies and model architectures through a coupled approach (i.e., *hardcoded*) by domain experts. (2) *XLA*[2] is an open-source machine learning compiler that transforms captured computation graphs into more efficient code through compilation techniques, efficiently overlapping computation and communication operations by locally adjusting operator order during the compilation process.

To ensure a fair evaluation and eliminate framework-specific implementation impacts, we capture the computation graphs of the parallel training frameworks without any optimizations and use them as input to the compiler-based approaches. Specifically, when testing LYNX and XLA, we capture the computation graphs from the Megatron-LM parallel training framework without any optimizations as input to the LYNX and XLA.

Performance Metrics. We primarily report (1) the Model FLOPs Utilization (MFU), which reflects system throughput and utilization of computational resources; and (2) the communication ratio, defined as the percentage of time spent on non-overlapped communication relative to the total time of computation and non-overlapped communication, which indicates the communication profile during training.

4.2 End-to-End Performance

In this section, we present the end-to-end performance results between LYNX and the state-of-the-art. We conduct tests on single-machine and multi-machine environments using various parallel strategies, including single parallel and hybrid parallel strategies, to further demonstrate the generality of LYNX. We use the following notation to represent the parallel strategies employed: 2DP $\times$ 8TP $\times$ 8EP indicates a hybrid parallel strategy using 16 GPUs, where the model is replicated twice (2DP), each replicated model is partitioned

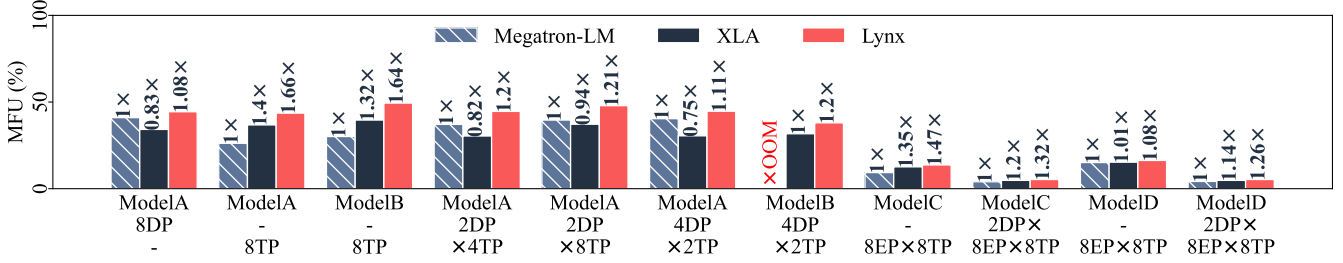


Figure 6. Model FLOPs Utilization (MFU).

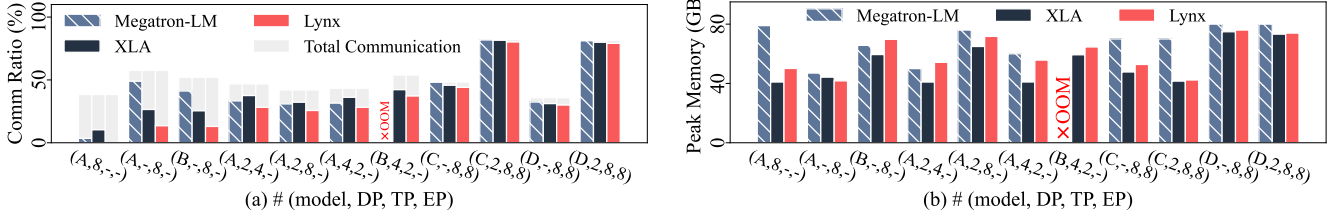


Figure 7. (a) Communication ratio, and (b) GPU peak memory. *Comm ratio* is the percentage of non-overlapped communication time relative to total computation and non-overlapped communication time (i.e., the lower the better). The *total communication* shows the proportion of all communication time (overlapped and non-overlapped) to total time (computation and all communication times). Here, (A, 8, -, -) indicates training ModelA with a DP degree set to 8.

across 8 GPUs (8TP), and with each GPU assigned 16/8 experts (8EP). It should be noted that our DP strategy includes ZeRO-3 (i.e., equivalent to FSDP).

Results in MFU and Communication Ratio: The performance results in Figure 6 indicate that LYNX achieves superior performance compared to the baseline when training different models using various parallel strategies.

When using only the DP strategy, LYNX achieves a performance improvement of 1.08 $\times$ over Megatron-LM. This is because, under this parallel strategy, simply using the expert-crafted *compute-time prefetching pattern* can effectively overlap computation and communication, thus leaving limited space for optimization. As shown in Figure 7(a), Megatron-LM has only approximately 3.5% non-overlapped communication time, yet LYNX can still identify these overlapping opportunities, achieving nearly 100% overlapping. XLA, due to prioritizing memory overhead during scheduling, exhibits less memory usage, as illustrated in Figure 7(b), and a lower overlapping ratio under this parallel strategy.

When using only the TP strategy, LYNX achieves performance improvements of 1.66 $\times$ over Megatron-LM and 1.2 $\times$ over XLA. This is because, under this parallel strategy, significant optimization opportunities exist that experts failed to identify from the local optimization view, while the methods advocated by XLA discovered some local overlapping opportunities from a fine-grained local optimization view. As shown in Figure 7(a), Megatron-LM exhibits a 49.0% non-overlapped communication ratio. Because of its heuristic scheduling algorithm, LYNX is able to reduce the non-overlapped communication ratio by 35.4%.

In more complex scenarios involving hybrid DP+TP parallel, LYNX also demonstrates significant advantages, achieving up to a 47% performance improvement (e.g., ModelC with 8EP $\times$ 8TP). In such hybrid parallel strategies, both expert-crafted methods and XLA’s local overlapping approaches struggle to identify additional opportunities for computation-communication overlapping. The performance advantages exhibited by LYNX in these scenarios underscore its high generality and performance.

When training larger-parameter models (ModelC and ModelD), the sparsity inherent in the MoE results in lower MFU compared to dense models (ModelA and ModelB). Specifically, in scenarios requiring inter-machine parameter synchronization (i.e., DP=2), as shown in Figure 7(a), the large volume of parameters requiring synchronization leads to communication delays accounting for 80% of the total latency, resulting in extremely low MFU. Even in such scenarios, LYNX still manages to find some overlapping opportunities, achieving higher performance (e.g., by up to 32% for ModelC 2DP $\times$ 8EP $\times$ 8TP) than the baselines.

Memory Usage. To verify that LYNX does not incur significant additional memory overhead due to overlapping computation and communication, we monitor the peak memory usage of the GPU in our experiments. The results are shown in Figure 7(b). In almost all scenarios, XLA uses the least memory, as its scheduling method is designed with minimal memory usage as the objective. Meanwhile, Megatron-LM and LYNX demonstrate comparable memory usage in almost all scenarios. This indicates that LYNX does not introduce additional memory overhead while improving performance.

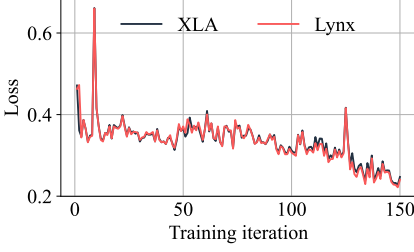


Figure 8. Loss of LYNX.

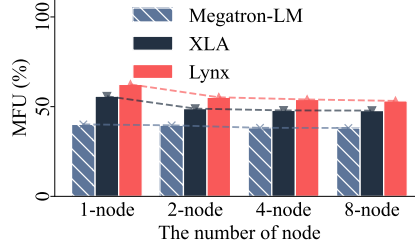


Figure 9. Scalability of LYNX.

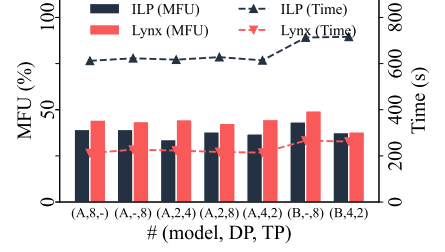


Figure 10. Efficiency of LYNX.

Model Convergence. We conduct full fine-tuning of ModelA using our in-house dataset to verify that training models with LYNX does not affect model performance. During training, we trace the loss values for each iteration with both XLA and LYNX. The results are presented in Figure 8. LYNX demonstrates loss values consistent with those of XLA, indicating that training models with LYNX does not negatively affect the final performance of models.

4.3 Performance Scalability

We employ a hybrid parallel strategy combining DP+TP to train ModelA on 1, 2, 4, and 8-node setups to demonstrate the scalability of LYNX. In this hybrid parallel strategy, we set the DP degree equal to the number of nodes and the TP degree equal to the number of GPUs within each node. This means the model is replicated across each machine, and within each machine, TP with a degree of 8 partitions the model across 8 GPUs. The experimental results are shown in Figure 9. Similar to XLA and Megatron-LM, LYNX exhibits a very slight downward trend during node scaling, but LYNX consistently outperforms them. This is because, under this hybrid parallel strategy, their communication-computation overlapping patterns remain essentially unchanged during node scaling. However, as the number of nodes increases, there is a minimal increase in communication latency, resulting in an extremely slight downward trend in MFU.

4.4 Efficiency of Heuristics Algorithm

To evaluate the efficiency of the heuristic algorithm proposed in §3.2, we replace LYNX’s heuristic algorithm for solving computation-communication overlapping patterns with Algorithm 1, which transforms the problem of finding optimal overlapping patterns into an integer linear programming (ILP) problem and solves it using Google’s OR-Tools 10 (an off-the-shelf solver). We then conduct end-to-end experiments comparing the MFU of overlapping patterns obtained by the heuristics algorithm and ILP solver. Additionally, since both LYNX and the ILP solver solve for the optimal solution iteratively, we record the time required to reach convergence, defined as the time point beyond which additional iterations or time do not yield a better overlapping pattern.

The results, as shown in Figure 10, demonstrate that the algorithm proposed by LYNX achieves a better overlapping pattern across various scenarios with complex constraints (i.e., complex computational graphs). OR-Tools required 10 minutes to solve the ILP problem, and additional time rarely produced a better overlapping pattern, whereas LYNX’s heuristic algorithm find a better overlapping pattern than the ILP in just 4 minutes. Furthermore, as the computational graph complexity increases (i.e., from ModelA to ModelB), solving time increases for both methods, but the ILP’s increase is significantly higher than LYNX’s, indicating LYNX’s lower time complexity. These results demonstrate that the heuristic algorithm proposed by LYNX obtains a superior overlapping pattern in less time compared to the ILP.

4.5 Effectiveness of Local Search Algorithm

We conduct end-to-end experiments comparing the MFU of overlapping patterns obtained with and without the *Local Search* algorithm for solving overlapping patterns. This comparison evaluates whether the second-phase *Local Search* algorithm in the heuristic algorithm, which performs local adjustments, can further obtain better overlapping patterns. Figure 11 demonstrates that without the second-phase *Local Search* algorithm, the MFU decreases by 1-2% in almost all scenarios, confirming that the *Local Search* algorithm, albeit slightly, can discover local overlapping opportunities overlooked by the first-phase *Tree Search* algorithm.

4.6 Effectiveness of SubGraph Rewrite

We conduct end-to-end experiments using MoE architecture models (ModelC and ModelD) to evaluate the effectiveness of the *subgraph rewrite* technique, comparing the time required to solve the overlapping pattern with and without the *subgraph rewrite* technique. We select MoE architecture models to validate the *subgraph rewrite* technique because, in these models, the gating mechanism during execution selects different *expert branches* (i.e., executing different control flow branches) due to random sampling operations, resulting in variations in the computational graph. Figure 12 shows that without the *subgraph rewrite* technique, the computational graph undergoes 10 changes during DL model training, requiring the overlapping pattern to be solved 10 times for the

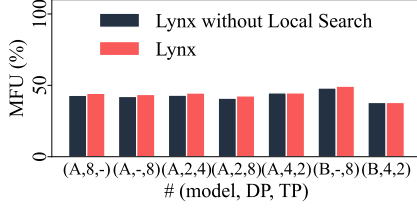


Figure 11. Effectiveness of Local Search Algorithm.

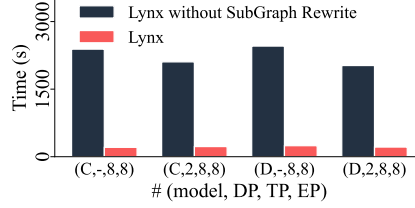


Figure 12. Effectiveness of SubGraph Rewrite.

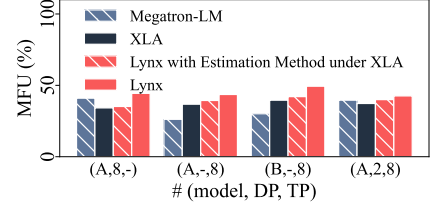


Figure 13. Effectiveness of Runtime Feedback.

entire graph, thus incurring an overhead of approximately 40 minutes. After applying the *subgraph rewrite* technique, solving the overlapping pattern for the entire graph is no longer necessary, adding only about 1 minute of additional solving time. Therefore, the *subgraph rewrite* technique reduces the overhead caused by computational graph changes.

4.7 Effectiveness of Runtime Feedback

We evaluate the effectiveness of the *runtime feedback* component, replacing the runtime feedback mechanism in LYNX with the XLA operator execution time estimation method, which relies on formulas to estimate operator execution overhead. We then conduct end-to-end experiments comparing the MFU achieved by these two approaches.

The experimental results, as shown in Figure 13, indicate that even without the runtime feedback mechanism, i.e., using XLA’s operator cost estimation method, LYNX still achieves superior results than XLA, further underscoring the strength of LYNX’s heuristic algorithm. However, the absence of accurate operator execution estimates provided by the runtime feedback mechanism leads to varying degrees of performance degradation, with almost all scenarios exhibiting a lower MFU than when runtime feedback is enabled. These results confirm the effectiveness of the runtime feedback mechanism.

5 Discussions and Future Work

Extending LYNX to overlap multiple operators. When solving for optimal overlapping patterns, LYNX’s modeling approach is limited to overlapping only a single communication operator with a single computation operator. However, extending LYNX to overlap multiple operators of the same or different types would be beneficial, similar to some research work [9, 24, 37, 40] that discusses how to overlap multiple computation operators for efficient GPU utilization.

Considering memory constraints. The scheduling strategy of LYNX focuses solely on how to overlap more communication with computation, without considering memory constraints. As shown in 7(b), this may increase peak memory usage in certain scenarios; thus, it remains important to treat memory as a critical resource in scheduling. In the future, we plan to incorporate memory limitations into the scheduling strategy to balance performance and memory.

6 Related Work

Communication in Distributed Training. Parallel strategies introduce communication operations on the critical path of computation, resulting in low MFU during training. Works such as Alpa [51] and others [21, 26, 32, 34, 35, 49, 53] improve parallel training efficiency by identifying optimal combinations among various parallel strategies. LYNX aiming to discover opportunities for computation-communication overlapping within parallel strategies, which can further optimize the efficiency of the parallel strategies they identify.

Computation-Communication Overlapping. Communication Overlapping is widely employed in distributed training to enhance system training efficiency by hiding communication overhead. Centauri [10] and others [20, 46, 47] decompose the operators into fine-grained operators to create more overlapping opportunities. Moreover, FLUX [8] and others [19, 38, 52], further improve the efficiency of concurrent computation and communication execution through techniques such as operator fusion. LYNX can be integrated with them to further discover overlapping opportunities from the search space they create.

Communication Overlapping Pattern. Many works [16, 20, 22, 29, 33, 36, 46] hide communication through expert-designed *pipeline overlapping* pattern, thus lacking generality. Other works [2, 10] adjust local regions greedily to overlap, thereby overlooking potential opportunities. LYNX models the search for optimal overlapping patterns as a resource-constrained scheduling problem and solves it through the heuristic algorithm, achieving generality and performance.

7 Conclusion

We present LYNX, a generic and efficient distributed training system designed to optimize the training efficiency of deep learning models by hiding communication through finding optimal computation-communication overlapping patterns. We formulate the problem of finding an optimal overlapping pattern as a resource-constrained scheduling problem. Based on our analysis of operator distributions in computation graphs for training different models with various parallel strategies, we propose a two-phase heuristic scheduling algorithm that efficiently solves this problem. Our comprehensive evaluation demonstrates that LYNX significantly improves GPU utilization compared to the state-of-the-art.

References

- [1] 2020. Collective Operations – NCCL 2.23.4 document. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>.
- [2] 2024. XLA. <https://github.com/openxla/xla>.
- [3] 2025. NVIDIA CUDA Profiling Tools Interface (CUPTI). <https://developer.nvidia.com/cupti>.
- [4] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Rafal Jozefowicz, Yangqing Jia, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Mike Schuster, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. *TensorFlow, Large-scale machine learning on heterogeneous systems*. <https://doi.org/10.5281/zenodo.4724125>
- [5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [6] Anirudha Agrawal, Shaizeen Aga, Suchita Pati, and Mahzabeen Islam. 2024. Optimizing ML Concurrent Computation and Communication with GPU DMA Engines. *arXiv preprint arXiv:2412.14335* (2024).
- [7] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.
- [8] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, et al. 2024. FLUX: fast software-based communication overlap on gpus through kernel fusion. *arXiv preprint arXiv:2406.06858* (2024).
- [9] Aodong Chen, Fei Xu, Li Han, Yuan Dong, Li Chen, Zhi Zhou, and Fangming Liu. 2024. Opara: Exploiting Operator Parallelism for Expediting DNN Inference on GPUs. *IEEE Trans. Comput.* (2024).
- [10] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 178–191.
- [11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [12] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [13] George B Dantzig. 2002. Linear programming. *Operations research* 50, 1 (2002), 42–47.
- [14] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [15] Weiqi Feng, Yangrui Chen, Shaoyu Wang, Yanghua Peng, Haibin Lin, and Minlan Yu. 2024. Optimus: Accelerating Large-Scale Multi-Modal LLM Training by Bubble Exploitation. *arXiv preprint arXiv:2408.03505* (2024).
- [16] Hao Ge, Junda Feng, Qi Huang, Fangcheng Fu, Xiaonan Nie, Lei Zuo, Haibin Lin, Bin Cui, and Xin Liu. 2025. ByteScale: Efficient Scaling of LLM Training with a 2048K Context Length on More Than 12,000 GPUs. *arXiv preprint arXiv:2502.21231* (2025).
- [17] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. 2017. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409* (2017).
- [18] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).
- [19] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 402–416.
- [20] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. 2024. Lancet: Accelerating Mixture-of-Experts Training via Whole Graph Computation-Communication Overlapping. *Proceedings of Machine Learning and Systems* 6 (2024), 74–86.
- [21] Youhe Jiang, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, and Bin Cui. 2022. Osdp: Optimal sharded data parallel for distributed deep learning. *arXiv preprint arXiv:2209.13258* (2022).
- [22] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 745–760.
- [23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [24] Woosuk Kwon, Gyeong-In Yu, Eunji Jeong, and Byung-Gon Chun. 2020. Nimble: Lightweight and parallel gpu task scheduling for deep learning. *Advances in Neural Information Processing Systems* 33 (2020), 8343–8354.
- [25] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
- [26] Dacheng Li, Hongyi Wang, Eric Xing, and Hao Zhang. 2022. Amp: Automatically finding model parallel strategies with heterogeneity awareness. *Advances in Neural Information Processing Systems* 35 (2022), 6630–6639.
- [27] Fanxin Li, Shixiong Zhao, Yuhao Qing, Xusheng Chen, Xiuxian Guan, Sen Wang, Gong Zhang, and Heming Cui. 2023. Fold3D: Rethinking and parallelizing computational and communicational tasks in the training of large DNN models. *IEEE Transactions on Parallel and Distributed Systems* 34, 5 (2023), 1432–1449.
- [28] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Transactions on Parallel & Distributed Systems* 32, 03 (2021), 708–727.
- [29] Shengwei Li, Kai Lu, Zhiqian Lai, Weijie Liu, Keshi Ge, and Dongsheng Li. 2024. A multidimensional communication scheduling method for hybrid parallel dnn training. *IEEE Transactions on Parallel and Distributed Systems* (2024).
- [30] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2023. Sequence Parallelism: Long Sequence Training from System

- Perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2391–2404.
- [31] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. [n. d.]. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment* 13, 12 ([n. d.]).
 - [32] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, Cheng Li, Saeed Maleki, Xu Cao, Ning Shang, Yilei Yang, et al. 2024. {nnScaler}:{Constraint-Guided} Parallelization Plan Generation for Deep Learning Training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 347–363.
 - [33] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
 - [34] Yuliang Liu, Shenggui Li, Jiarui Fang, Yanjun Shao, Boyuan Yao, and Yang You. 2023. Colossal-auto: Unified automation of parallelization and activation checkpoint for large-scale models. *arXiv preprint arXiv:2302.02599* (2023).
 - [35] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient transformer training over multiple gpus using automatic parallelism. *arXiv preprint arXiv:2211.13878* (2022).
 - [36] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
 - [37] Jongseok Park, Kyungmin Bin, Gibum Park, Sangtae Ha, and Kyung-han Lee. 2023. Aspen: Breaking operator barriers for efficient parallelization of deep neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 68625–68638.
 - [38] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1146–1164.
 - [39] Laurent Perron and Frédéric Didier. [n. d.]. CP-SAT. Google. https://developers.google.com/optimization/cp/cp_solver/
 - [40] Yukai Ping, He Jiang, Xingxiang Liu, Zhenyang Zhao, Zhide Zhou, and Xin Chen. 2023. Latency-Based Inter-Operator Scheduling for CNN Inference Acceleration on GPU. *IEEE Transactions on Services Computing* 17, 1 (2023), 277–290.
 - [41] Francesca Rossi, Peter Van Beek, and Toby Walsh. 2008. Constraint programming. *Foundations of Artificial Intelligence* 3 (2008), 181–211.
 - [42] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).
 - [43] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
 - [44] H. Topcuoglu, S. Hariri, and Min-You Wu. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274. <https://doi.org/10.1109/71.993206>
 - [45] Jeffrey D. Ullman. 1975. NP-complete scheduling problems. *Journal of Computer and System sciences* 10, 3 (1975), 384–393.
 - [46] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).
 - [47] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. 2022. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 93–106.
 - [48] Ruisi Zhang, Tianyu Liu, Will Feng, Andrew Gu, Sanket Purandare, Wanchao Liang, and Francisco Massa. 2024. SimpleFSDP: Simpler Fully Sharded Data Parallel with torch. compile. *arXiv preprint arXiv:2411.00284* (2024).
 - [49] Shiwei Zhang, Lansong Diao, Siyu Wang, Zongyan Cao, Yiliang Gu, Chang Si, Ziji Shi, Zhen Zheng, Chuan Wu, and Wei Lin. 2023. Auto-parallelizing large models with rhino: A systematic approach on production ai platform. *arXiv preprint arXiv:2302.08141* (2023).
 - [50] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3848–3860.
 - [51] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
 - [52] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, et al. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. *arXiv preprint arXiv:2503.20313* (2025).
 - [53] Yonghao Zhuang, Lianmin Zheng, Zhuohan Li, Eric Xing, Qirong Ho, Joseph Gonzalez, Ion Stoica, Hao Zhang, and Hexu Zhao. 2023. On optimizing the communication of model parallelism. *Proceedings of Machine Learning and Systems* 5 (2023), 526–540.